```
  else
    element.className = newClass
```

Try to write a test for it:

```
fact "addClass adds a class to an element using a selector", ->
  addClass '#a .b', 'popular'
  actualClass = document.querySelector(selector).className
  assert.equals 'product special popular', actualClass
```

This is actually asserting the result of a side effect. That smells.

Notice that the test asserts something on the `document`. This is only *indirectly* testing the `addClass` function. Worse, if the `document` ever changes the way it works, you'll need to update both the function and the test. No thanks.

The `addClass` function depends on the presence of a `document` to work. In other words, `document` is a dependency. How do you deal effectively with dependencies in a test?

10.3. Dependencies

When you write a test, the part of the program you're testing is the *system under test*. Anything that you're not testing but that the system under tests *uses* is a dependency.

Imagine a car. To test if the battery is working, a device called an *ammeter*, which tests if the battery is producing electricity, is used. The car needs electricity to start and to power accessories such as the radio. A battery that's removed from the car can still be tested with an ammeter. Not the radio, though; if the battery is removed, then the radio won't turn on, preventing it from being tested. How do you test the radio without the battery? Well, the radio doesn't need the battery *specifically* but only some equivalent source of electricity.

The same principle applies to components in a program. When you test part of a program, the dependencies don't need to be the real thing, just something that's equivalent as far as the test is concerned. In this section